The Design of an Online Library Database System: Multi-Library Management

13 min read · Jan 2, 2024

Laela Hajat Maqbullah Follow

Listen Share

Libraries play a crucial role as places for information and learning, offering a diverse range of books for people to explore. However, behind this noble function, libraries often face challenges in managing data, including recording books, members, borrowing, returning, and generating reports. The prevalent manual processes can lead to data damage, writing errors, or even crucial information loss.

Aligned with technological advancements, this project aims to design and implement a comprehensive and efficient database system for an online library application. The system will serve as a strong foundation for effective data management, emphasizing improvements in efficiency and user experience for the electronic library application.

To accomplish this task, I will utilize the PostgreSQL application and SQL (Structured Query Language). SQL, as a programming language specialized in managing data within a Relational Database Management System (RDBMS), will facilitate the execution of necessary commands for book, member, borrowing, and storage data management. The RDBMS, as a supporting software, enables efficient editing, changes, additions, and updates to data.

By implementing this database system, the aim is to enhance the quality of services offered by the online library, ensuring data integrity and providing better tools for tracking book transactions. Moreover, the choice of PostgreSQL and SQL as the primary platforms brings advantages in terms of reliability and performance, supporting the vision to create an efficient and trustworthy online library.

Project's Objectives

The primary objective of this project is to design and implement a comprehensive and efficient database system for an online library application. The system aims to manage multiple libraries, diverse book collections, user registrations, and a robust loan and hold system. The design should provide a foundation for effective data management, improving the overall efficiency and user experience of the e-library application.

Designing The Database

1. Mission Statement

The mission of the e-library database is to create a robust system that supports managing multiple libraries, diverse book collections, and user interactions. The system ensures efficient book borrowing, automatic returns, and a streamlined hold system.

o **Tables Description**

Tables	Descriptions
libraries	This table holds information about the libraries in the e-library system, including unique identifiers (library_id) and names (library_name). It helps in managing multiple libraries.
genres	This table manages different genres of books available in the e-library. It includes unique identifiers (genre_id) and names (genre_name).
books	The core table for book-related information. It includes details such as title (title), author (author), available quantity (quantity), and genre (genre_id). The library_id field connects books to specific libraries.
library_books	This table represents the relationship between libraries and books. It includes a unique identifier (lib_book_id) and foreign keys (library_id and book_id) to establish connections between libraries and their respective book collections.
Users	This table stores information about registered users on the e-library platform, including unique identifiers (user_id), names (first_name and last_name), contact details (email and phone_number), and their interactions with the system.
loan	This table keeps track of loan transactions. It includes unique identifiers (loan_id), foreign keys (user_id and book_id) linking to users and books, and relevant loan details such as dates (loan_date, due_date, and return_date) and loan status (loan_status).
hold	The hold table manages information related to holds placed by users on books. It includes unique identifiers (hold_id), foreign keys (user_id and book_id) connecting to users and books, and relevant hold details such as dates (hold_date and release_date) and hold status (status).

image by author

o **Limitations for an e-library application:**

· Implement constraints to ensure users can only borrow two books at a time and store a maximum of two books simultaneously.

· Ensure the borrowing period is enforced through the Due Date column in the Transactions table.

· Implement automation to manage automatic book returns when exceeding the due date.

· Create a mechanism to release stored books so that other users can borrow them if the customer does not borrow them within one week.

· This design provides the foundation for managing key features of the online library application, including book collections, user registration, borrowing and storage systems, as well as effective tracking of transactions in a multi-library environment.

2. Creating Table Structures

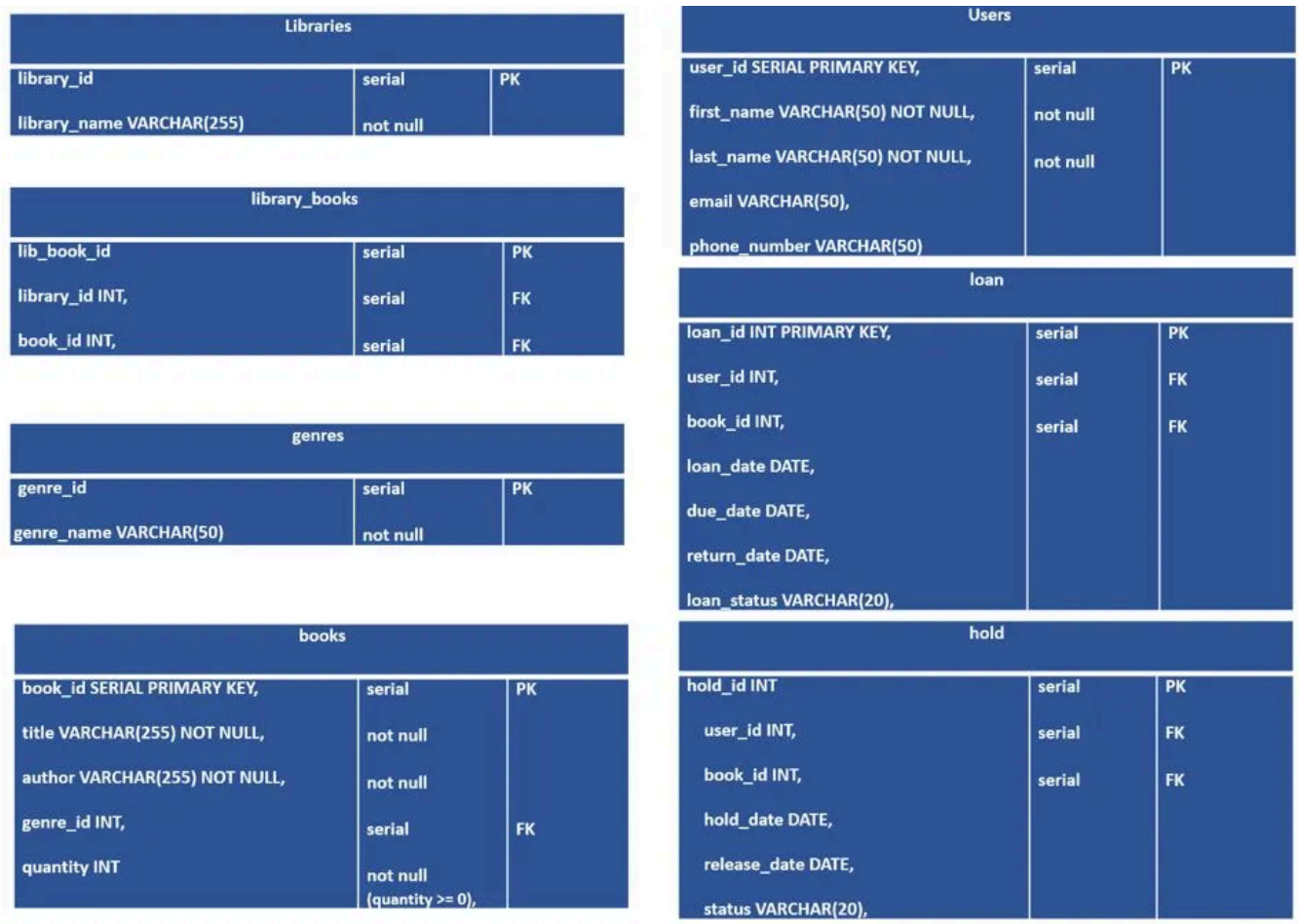


image by author

3. Determine Table Relationships:

One-to-Many Relationship:

· libraries to library books (One library can have many books).

· genres to books (One genre can be associated with many books).

· users to loans (One user can have multiple loans).

· users to holds (One user can have multiple holds).

Many-to-Many Relationship:

· books to libraries (A book can be in multiple libraries, and a library can have multiple books).

· users to books (A user can borrow multiple books, and a book can be borrowed by multiple users).

4. Determine Business Rules:

a. Libraries Table:

• library_id: Unique identifier for each library.

• library_name: NOT NULL — Ensure that each library has a name.

b. Genres Table:

• genre_id: Unique identifier for each genre.

• genre_name: NOT NULL — Make sure each genre has a name.

c. Books Table:

• book_id: Unique identifier for each book.

• title: NOT NULL — Ensure that each book has a title.

• author: NOT NULL — Ensure that each book has an author.

• genre_id: Foreign key referencing the Genres table.

• quantity: NOT NULL, CHECK(quantity >= 0) — Quantity should not be negative.

d. Library Books Table (Join Table):

• lib_book_id: Unique identifier for each library-book relationship.

• library_id: NOT NULL — Ensure that each library book is associated with a library (Foreign key referencing the Libraries table)

• book_id: NOT NULL — Ensure that each library book is associated with a book (Foreign key referencing the Books table).

e. Users Table

• user_id: Unique identifier for each user

• first_name: NOT NULL — Users must have a first name.

• last_name: NOT NULL — Users must have a last name.

• email: UNIQUE — Ensure that each user has a unique email.

• phone_number: UNIQUE — Ensure that each user has a unique phone number.

f. Loan Table:

• loan_id: Unique identifier for each loan

• user_id: Foreign key referencing the Users table.

• book_id: Foreign key referencing the Books table

• loan_date: NOT NULL — Record the date when a book is borrowed.

• due_date: NOT NULL — Record the due date for book return.

• return_date — Allow NULL, as the book may not have been returned yet.

• loan_status: CHECK (loan_status IN (‘on loan’, ‘Overdue’/it has returned) — Ensure that the loan status is valid.

• Constraint: loan_date must be before due_date

g. Hold Table:

• hold_id: Unique identifier for each hold.

• user_id: Foreign key referencing the Users table.

• book_id: Foreign key referencing the Books table

• hold_date: NOT NULL — Record the date when a user places a hold.

• release_date — Allow NULL, as the book may not have been released from hold.

- status: CHECK status IN ‘In Queue’ — Ensure that the hold status is valid.

5. Implementing The Design

Design ERD

The Entity-Relationship Diagram (ERD) illustrates the relationships between key entities in the e-library application. The design includes tables for libraries, genres, books, library-books relationships, users, loans, and holds. Relationships are well-defined using primary and foreign keys, ensuring data integrity and efficient data retrieval. This ERD design uses the draw.io tool:

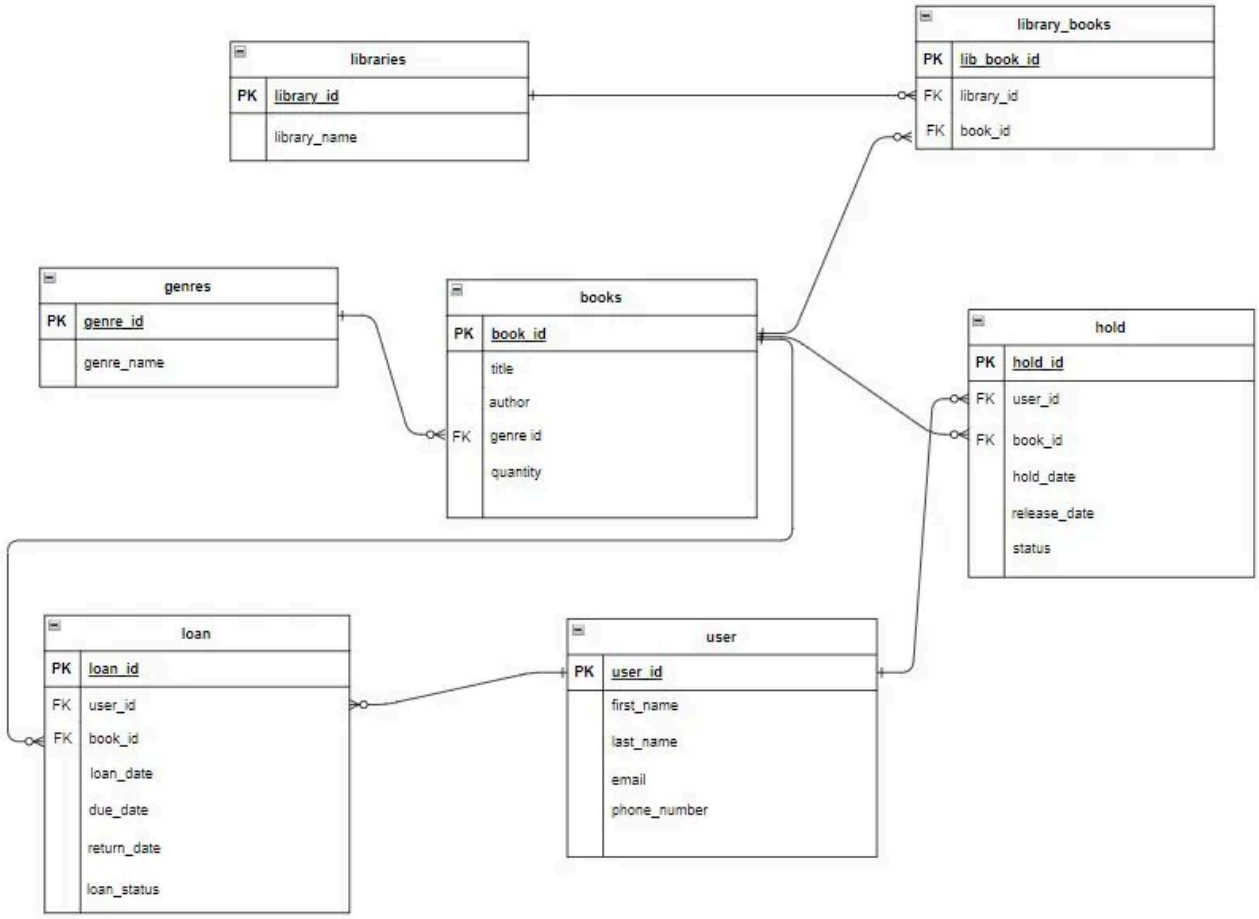


image by author

Implementation of the design or Entity-Relationship Diagram (ERD) in a PostgreSQL database involves several steps as follows:

1. Create Database

-Open pgAdmin and navigate to the Databases section.

-Right-click and select “Create Database.”

-Provide the name as “E_library.”

Begin by creating a new database in PostgreSQL that will accommodate the tables designed in the ERD.

2. Create Table

The design has been implemented in PostgreSQL using Data Definition Language (DDL) syntax. The database includes tables for libraries, genres, books, library-books relationships, users, loans, and holds. Foreign key constraints have been applied to maintain referential integrity between tables.

DDL:

```
CREATE TABLE libraries (
    library_id SERIAL PRIMARY KEY,
    library_name VARCHAR(255) NOT NULL
);
```

```
CREATE TABLE genres (
    genre_id SERIAL PRIMARY KEY,
    genre_name VARCHAR(50) NOT NULL
);
```

```
CREATE TABLE books (
    book_id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(255) NOT NULL,
    genre_id INT,
    quantity INT NOT NULL CHECK (quantity >= 0),
    FOREIGN KEY (genre_id) REFERENCES genres(genre_id)
);
```

```
CREATE TABLE library_books (
    lib_book_id INT PRIMARY KEY,
    library_id INT,
    book_id INT,
    FOREIGN KEY (library_id) REFERENCES libraries(library_id),
    FOREIGN KEY (book_id) REFERENCES books(book_id)
);
```

```
CREATE TABLE user (
    user_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(50),
    phone_number VARCHAR(50)
);
```

```
CREATE TABLE loan (
    loan_id INT PRIMARY KEY,
    user_id INT,
    book_id INT,
    loan_date DATE,
    due_date DATE,
    return_date DATE,
    loan_status VARCHAR(20),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (book_id) REFERENCES books(book_id),
    CONSTRAINT loan_check CHECK (loan_date < due_date)
);
```

```
CREATE TABLE hold (
    hold_id INT PRIMARY KEY,
    user_id INT,
    book_id INT,
    hold_date DATE,
    release_date DATE,
    status VARCHAR(20),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (book_id) REFERENCES books(book_id)
);
```

image by author

The following ERD is generated based on the tables that have been designed and created in PostgreSQL:

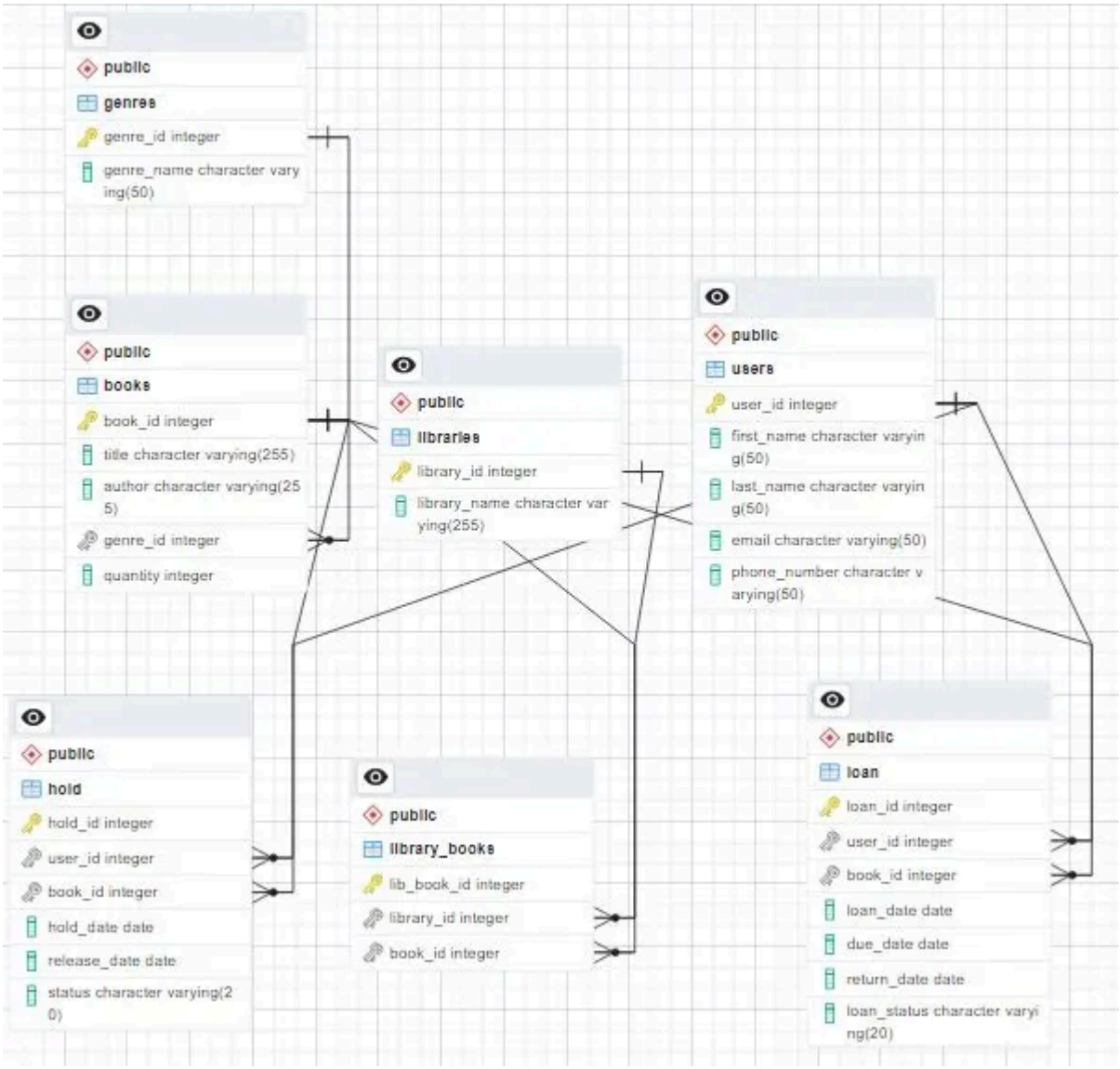


image by author

Populating the Database

1. Create Dummy Dataset

A dummy dataset has been generated using Python to simulate realistic data for each table. The dataset includes information about users, libraries, library books, genres, books, loan, and hold.

1. Instal faker and tabulate

```
!pip install Faker
!pip install tabulate
```

image by author

2. Import library

```
# Import Library
from faker import Faker
from tabulate import tabulate
import random
from datetime import datetime, timedelta
import csv
```

image by author

3. Display data in table form

```
def show_data(table):

    tab = tabulate(tabular_data = table,
                   headers = table.keys(),
                   tablefmt = "psql",
                   numalign = "center")
    print(tab)
```

image by author

4. Create dummy data for **user tables** and save the data in CSV form

Generates dummy data for the user table which contains user_id, first_name, last_name, email and phone number.

```python
import pandas as pd
from faker import Faker

# Create a Faker instance with Indonesian locale
FAKER = Faker('id_ID')

# Function to generate dummy data for the user table
def generate_name(n):
    return [FAKER.name() for _ in range(n)]

# Function to display the generated table
def show_data(table):
    df = pd.DataFrame(table)
    print(df)

# Function to generate dummy data for the user table
def user_table(n_user, is_print, file_path='tab_user.csv'):
    # Initialize an empty dictionary to store user data
    table = {}

    # Assign user_id sequentially starting from 1
    table["user_id"] = [i + 1 for i in range(n_user)]

    # Generate random names using the Faker library with Indonesian locale
    names = generate_name(n_user)
    # Extract first names and last names
    table['first_name'] = [i.split(' ')[0] for i in names]
    table['last_name'] = [i.split(' ')[1] for i in names]

    # Generate random email addresses based on the generated names
    table['email'] = [f"{name.lower().replace(' ', '')}@{FAKER.free_email_domain()}" for name in names]

    # Generate random phone numbers
    table['phone_number'] = [FAKER.phone_number() for _ in range(n_user)]

    # Convert the dictionary to a DataFrame
    user_df = pd.DataFrame(table)

    # Print the generated table
    if is_print:
        show_data(table)

    # Save the DataFrame to a CSV file
    user_df.to_csv(file_path, index=False)

    return user_df

# Example usage:
# Generate a table of 500 users with Indonesian names, display it, and save it to a CSV file
user_data = user_table(n_user=500, is_print=True, file_path='tab_user.csv')
```

image by auhtor

5. Create dummy data for libraries tables and save the data in CSV form

Generates dummy data for the libraries table which contains library_id, and library_name.

```python
import pandas as pd
from faker import Faker

fake = Faker()

# Function to generate dummy data for the Library table
def generate_library_data(num_records, is_print=False, file_path='tab_libraries.csv'):
    library_names = [f"Library {chr(65 + i)}" for i in range(num_records)]  # Generate names like "Library A", "Library B", ...
    data = {
        'library_id': range(1, num_records + 1),
        'library_name': library_names
    }
    library_table = pd.DataFrame(data)

    # Print table
    if is_print:
        show_data(library_table)

    # Save DataFrame to CSV
    library_table.to_csv(file_path, index=False)

    return library_table

# Function to display table
def show_data(table):
    print(tabulate(table, headers='keys', tablefmt='fancy_grid'))

# Example usage:
library_data = generate_library_data(num_records=5, is_print=True, file_path='tab_libraries.csv')
```

image by author

6. Create dummy data for library_books tables and save the data in CSV form.

Generates dummy data for the library_books table which contains lib_book_id, library_id, and book_id.

```python
import pandas as pd
import random
from faker import Faker
from tabulate import tabulate

fake = Faker()

# Function to generate dummy data for the library_books table
def generate_library_books_data(num_records, num_libraries, num_books):
    data = {
        'lib_book_id': range(1, num_records + 1),
        'library_id': [random.randint(1, num_libraries) for _ in range(num_records)],
        'book_id': list(range(1, num_records + 1))
    }
    return pd.DataFrame(data)

# Function to display table
def show_data(table):
    print(tabulate(table, headers='keys', tablefmt='fancy_grid'))

# Set the number of records you want in your dummy dataset for library_books
num_records_library_books = 500

# Assuming you have 5 libraries (A to E) and 500 books, adjust as needed
num_libraries = 5
num_books = 500

# Generate the library_books table with dummy data
library_books_data = generate_library_books_data(num_records_library_books, num_libraries, num_books)

# Display the generated library_books table using tabulate
show_data(library_books_data)

# Save the library_books_data DataFrame to a CSV file
library_books_data.to_csv('tab_book_library.csv', index=False)
```

image by author

7. Create dummy data for genres tables and save the data in CSV form

Generates dummy data for the genres table which contains genre_id and genre_name

```python
import random
import pandas as pd
from tabulate import tabulate

# Function to display table
def show_data(table):
    print(tabulate(table, headers='keys', tablefmt='fancy_grid'))

def genres_table(n_genres, is_print):
    # List of genres
    generate_genres = ['Fantasy', 'History', 'Thriller', 'Science Fiction', 'Biography', 'Romance', 'Mystery', 'Adventure']

    # Ensure n_genres does not exceed the length of generate_genres
    n_genres = min(n_genres, len(generate_genres))

    # Create a table
    table = {}
    table["genre_id"] = [i+1 for i in range(n_genres)]
    table['genre_name'] = random.sample(generate_genres, n_genres)

    # Print table
    if is_print:
        show_data(table)

    return table

# Set the number of genres you want in your dummy dataset
num_genres = 8

# Generate the genres table with dummy data
genres_data = genres_table(n_genres=num_genres, is_print=True)

# Save DataFrame to CSV with the desired file name
pd.DataFrame(genres_data).to_csv('tab_genres.csv', index=False)
```

image by author

8. Create dummy data for books tables and save the data in CSV form

Generates dummy data for the books table which contains book_id, title, author, genre_id, and quantity. Title and author data are taken from Kaggle data.

```python
import random
import pandas as pd

def books_table(n_books, is_print, file_path='goodreads_data.csv', csv_filename='tab_book.csv'):
    # Read data from CSV into a DataFrame
    df = pd.read_csv(file_path, encoding='utf-8')

    # Remove whitespaces from column names
    df.columns = df.columns.str.strip()

    # Get unique data from the 'Author' and 'Book' columns in the DataFrame
    unique_authors = df['Author'].unique()
    unique_books = df['Book'].dropna().unique()

    # Ensure that n_books does not exceed the length of unique_books
    n_books = min(n_books, len(unique_books))

    # Create a table dictionary
    table = {}
    # Assign sequential book_id starting from 1
    table['book_id'] = list(range(1, n_books + 1))

    # Select titles and authors simultaneously from unique data
    selected_books_authors = random.sample(list(zip(unique_books, unique_authors)), n_books)
    table['title'], table['author'] = zip(*selected_books_authors)

    # Generate random genre_id and quantity for each book
    table['genre_id'] = [random.randint(1, 8) for _ in range(n_books)]
    table['quantity'] = [random.randint(1, 5) for _ in range(n_books)]

    # Print the table
    if is_print:
        show_data(table)

    # Save DataFrame to CSV with the desired file name
    pd.DataFrame(table).to_csv(csv_filename, index=False)

    print(f"Data has been saved to '{csv_filename}'")

    return table

# Additional function to display data
def show_data(data):
    df = pd.DataFrame(data)
    print(df)

# Example usage:
# Generate a table of 500 books, display it, and save it to a CSV file
books_data = books_table(n_books=500, is_print=True, file_path='goodreads_data.csv', csv_filename='tab_book.csv')
```

9. Create dummy data for loan tables and save the data in CSV form

Generates dummy data for the loan table which contains loan_id, user_id, book_id, loan_date, due_date, return_date, and loan status.

```python
import random
from faker import Faker
from datetime import timedelta
import pandas as pd

fake = Faker()

def loan_table(n_loans, is_print, file_path='tab_loan.csv'):
    loan_data = []
    user_loan_count = {}

    def can_loan_more_books(user_id):
        return user_loan_count.get(user_id, 0) < 2

    for i in range(n_loans):
        user_id = random.randint(1, 500)
        book_id = random.randint(1, 500)

        if can_loan_more_books(user_id):
            loan_status = random.choice(['On Loan', 'Returned', 'Overdue'])

            user_loan_count[user_id] = user_loan_count.get(user_id, 0) + 1

            loan_data.append({
                'loan_id': i + 1,
                'user_id': user_id,
                'book_id': book_id,
                'loan_date': fake.date_time_between(start_date='-30d', end_date='now', tzinfo=None).date(),
                'due_date': None,
                'return_date': None,
                'loan_status': loan_status,
            })

    # Calculate due_date as 14 days from loan_date
    for entry in loan_data:
        entry['due_date'] = entry['loan_date'] + timedelta(days=14)

    # Calculate return_date based on loan status
    for entry in loan_data:
        if entry['loan_status'] in ['Returned', 'Overdue']:
            entry['return_date'] = fake.date_time_between(start_date=entry['loan_date'], end_date='now', tzinfo=None).date()

    # Convert to DataFrame
    loan_df = pd.DataFrame(loan_data)

    # Save DataFrame to CSV
    loan_df.to_csv(file_path, index=False)

    # Print generated data for verification
    if is_print:
        print(loan_df)

    return loan_df

# Example usage:
loan_data = loan_table(300, is_print=True, file_path='tab_loan.csv')
```

image by author

10. Create dummy data for hold tables and save the data in CSV form.

Generates dummy data for the hold table which contains hold_id, user_id, book_id, hold_date, release_date, and status.

```python
import random
from faker import Faker
from datetime import timedelta
import pandas as pd

fake = Faker()

def hold_table(n_entries, is_print, file_path='tab_hold.csv'):

    hold_data = []
    user_holds_count = {}

    def can_place_hold(user_id):
        return user_holds_count.get(user_id, 0) < 2

    for i in range(n_entries):
        user_id = random.randint(1, 500)
        book_id = random.randint(1, 500)

        if can_place_hold(user_id):
            user_holds_count[user_id] = user_holds_count.get(user_id, 0) + 1

            hold_data.append({
                'hold_id': i + 1,
                'user_id': user_id,
                'book_id': book_id,
                'hold_date': fake.date_time_between(start_date='-30d', end_date='now', tzinfo=None).date(),
                'release_date': fake.date_time_between(start_date='now', end_date='+7d', tzinfo=None).date(),
                'status': 'In Queue'
            })

    # Convert to DataFrame
    hold_df = pd.DataFrame(hold_data)

    # Save DataFrame to CSV
    hold_df.to_csv(file_path, index=False)

    # Print generated data for verification
    if is_print:
        print(hold_df)

    return hold_df

# Example usage:
hold_data = hold_table(50, is_print=True, file_path='tab_hold.csv')
```

image by author

Output=> Dataset in csv format:

· tab_user.csv

· tab_libraries.csv

· tab_book_library.csv

· tab_genres.csv

· tab_book.csv

· tab_loan.csv

· tab_hold.csv

2. Input Dummy Dataset into the Database

The dummy dataset has been successfully imported into the PostgreSQL database using the pgAdmin import data function. This ensures that the database is populated with realistic data for further analysis.

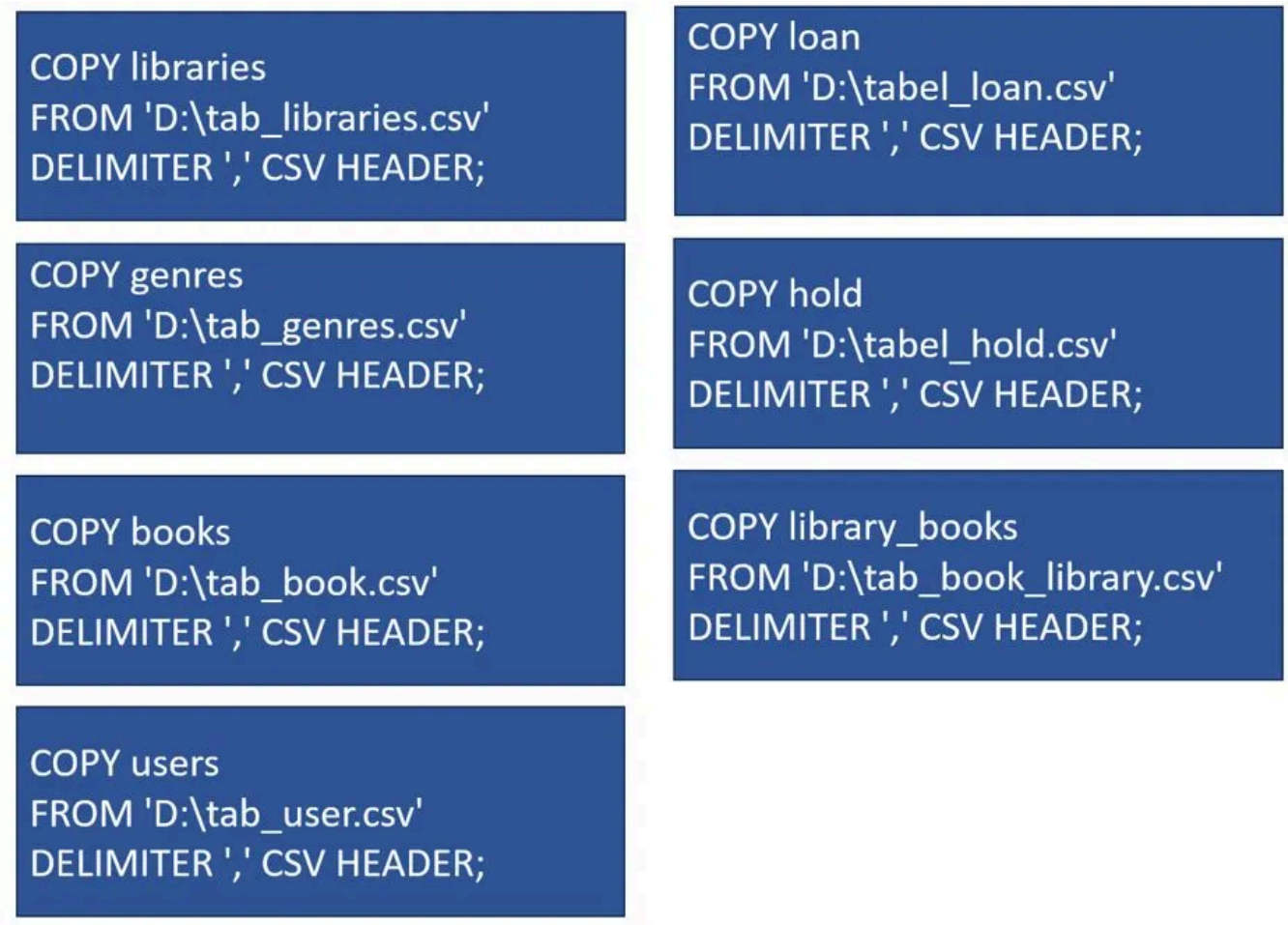


image by author

Retrieve Data

Write 5 Objectives/Questions

1. Determine the most popular genre based on the number of loans?

Understanding the popularity of genres based on loan statistics can guide the library or e-library platform in making informed decisions about collection development. It enables them to focus on acquiring more books in genres that are currently in high demand, ultimately enhancing user satisfaction and engagement.

○ SQL Query Syntax

```
SELECT genres.genre_name, COUNT(loan.loan_id) AS total_loans
FROM genres
LEFT JOIN books ON genres.genre_id = books.genre_id
LEFT JOIN loan ON books.book_id = loan.book_id
GROUP BY genres.genre_name
ORDER BY total_loans DESC;
```

image by author

○ Screenshot of Query Results:

	genre_name character varying (50)	total_loans bigint
1	Fantasy	43
2	Adventure	42
3	Romance	40
4	Biography	37
5	History	37
6	Thriller	35
7	Science Fiction	31
8	Mystery	26

image by author

Insights:

· Fantasy appears to be the most popular genre, with 43 total loans, indicating a high level of interest and demand among users. By knowing that this genre is highly sought after, e-library platforms can enrich their book collections in this genre.

· Adventure closely follows with 42 total loans, suggesting it is also a highly sought-after genre.

· Romance is in third place with 40 total loans, indicating a significant interest in romantic novels.

· Biography and History share a similar popularity level with 37 total loans each, showcasing interest in real-life stories and historical content.

· Thriller holds a respectable position with 35 total loans, reflecting the demand for suspenseful and thrilling narratives.

· Science Fiction follows with 31 total loans, indicating a decent level of interest in futuristic and imaginative storytelling.

· Mystery rounds out the list with 26 total loans, suggesting a moderate level of interest in mystery novels.

2. Titles of books that have been borrowed the most based on previous loan data (top 5)?

The purpose of this question is to provide insights to the library regarding the most borrowed books. The results show popular titles and offer practical suggestions, such as ensuring stock availability, exploring digital versions, and guiding collection development strategies based on reader preferences. This helps the library understand user interests and enhance satisfaction by providing sought-after books.

○ SQL Query Syntax

```sql
SELECT books.title, COUNT(loan.loan_id) AS total_loans
FROM books
LEFT JOIN loan ON books.book_id = loan.book_id
GROUP BY books.title
ORDER BY total_loans DESC
LIMIT 5;
```

image by author

○ Screenshot of Query Results:

	title character varying (255)	total_loans bigint
1	Charlotte's Web	4
2	One Thousand Gifts: A Dare to Live Fully Right Where You Are	4
3	Shot to Pieces	3
4	Song of Solomon	3
5	Presentation Zen: Simple Ideas on Presentation Design and Deliv...	3

image by author

Insights:

· “Charlotte’s Web” and “One Thousand Gifts: A Dare to Live Fully Right Where You Are” are the most borrowed books, each with 4 loans. This suggests a consistent interest in these titles among library users.

· “Shot to Pieces,” “Song of Solomon,” and “Presentation Zen: Simple Ideas on Presentation Design and Delivery” follow closely with 3 loans each. These books also demonstrate popularity among readers.

· The distribution of loan counts provides valuable insights into the reading preferences of library users. It indicates that certain books have garnered more attention and engagement, possibly due to their content, authorship, or relevance to the audience.

· Libraries or e-library platforms can capitalize on the popularity of these books by ensuring an adequate supply, considering potential additional copies, or exploring digital versions to meet the demand effectively.

3. Which books currently have the highest number of hold requests based on the hold queue data?

The purpose of this question is to provide insights to the library regarding books that currently have the highest number of hold requests based on hold queue data. The query below aims to identify books that are most in demand by library visitors according to the hold queue.

○ SQL Query Syntax

```sql
SELECT b.book_id, b.title, g.genre_name, COUNT(h.book_id) AS hold_count
FROM books b
JOIN genres g ON b.genre_id = g.genre_id
JOIN hold h ON b.book_id = h.book_id
GROUP BY b.book_id, b.title, g.genre_name
ORDER BY hold_count DESC
LIMIT 5;
```

image by author

○ Screenshot of Query Results:

	book_id integer	title character varying (255)	genre_name character varying (50)	hold_count bigint
1	234	Quantum Roots (Quantum Roots, #1)	Fantasy	2
2	62	Invisible Monsters	Adventure	2
3	161	Recursion	Thriller	2
4	289	World Peace: The Voice of a Mountain Bird	Mystery	1
5	255	Hoot	Adventure	1

Insights:

· “Quantum Roots (Quantum Roots, #1)” and “Invisible Monsters” are fantasy and adventure books, respectively, each with 2 hold requests. This suggests a notable interest in these genres, and these specific titles are in high demand among readers.

· “Recursion,” categorized as a thriller, also has 2 hold requests. This indicates a considerable demand for suspenseful and thrilling narratives among library users.

· “World Peace: The Voice of a Mountain Bird” falls under the mystery genre with 1 hold request. While it has a lower hold count compared to the others, it still signifies an audience seeking mystery-themed literature.

· “Hoot,” an adventure book, has 1 hold request. Similar to the other adventure book mentioned, it indicates a consistent interest in this genre.

· The distribution of hold requests across different genres provides insights into the specific genres that currently capture the attention and interest of library users.

· Libraries can use this information to optimize their collection by ensuring an adequate supply of books in these popular genres. They may consider acquiring additional copies, offering promotions, or implementing strategies to reduce wait times for these sought-after titles.

4. **How many users are still in the queue for book holds in December based on the hold queue data?**

The purpose of this question is to provide insights to the library about the number of users still in the queue for book holds in December based on hold queue data. The query below is designed to identify how many users are waiting for book holds in that specific month.

○ SQL Query Syntax

```
SELECT COUNT(DISTINCT user_id) AS users_in_queue
FROM hold
WHERE EXTRACT(MONTH FROM hold_date) = 12;
```

image by author

Screenshot of Query Results:

	users_in_queue bigint
1	48

image by author

Insight:

· 48 users in the hold queue in December suggest a notable level of demand for books during the holiday season or year-end activities. This heightened interest may be influenced by individuals seeking specific reading materials for leisure or gifts during this festive period.

· The library can use this information to optimize its book stocking strategy, ensuring an adequate supply of popular titles or genres during December to meet the increased demand.

· The substantial number of users in the hold queue implies that certain books or genres may be particularly sought after during December. Libraries could consider promoting these popular titles or organizing themed events to engage the community.

· Understanding the December hold queue helps the library efficiently allocate resources, including staff and infrastructure, to manage the increased demand and provide a satisfactory user experience.

· Libraries might want to implement targeted promotions, such as holiday-themed reading lists or special offers, to capitalize on the heightened interest in books during this festive season.

meet the specific demands of its users during this festive period.

5. **Which libraries have the highest circulation based on recent loan data?**

The goal of this question is to gain insights into which libraries have the highest circulation based on recent loan data. The provided query is designed to identify and rank libraries based on the count of loans, allowing for a comprehensive understanding of library usage.

○ SQL Query Syntax

```
SELECT l.library_id, l.library_name, COUNT(lo.loan_id) AS circulation_count
FROM libraries l
LEFT JOIN library_books lb ON l.library_id = lb.library_id
LEFT JOIN loan lo ON lb.book_id = lo.book_id
GROUP BY l.library_id, l.library_name
ORDER BY circulation_count DESC
LIMIT 5;
```

image by author

○ Screenshot of Query Results:

	library_id [PK] integer	library_name character varying (255)	circulation_count bigint
1	4	Library D	76
2	1	Library A	63
3	2	Library B	55
4	5	Library E	53
5	3	Library C	44

image by author

Insights:

· Library D has the highest circulation with 76 loans, indicating it is a highly utilized library. This may suggest a diverse and popular collection that caters to a broad audience.

· Library A follows closely with 63 loans, showcasing a strong engagement with library resources. The popularity of this library suggests it may offer a compelling selection of books or services.

· Library B, with 55 loans, demonstrates a substantial circulation, reinforcing its importance as a valuable resource for the community. This suggests a consistent demand for books available at this library.

· Library E and Library C have circulation counts of 53 and 44 loans, respectively. While slightly lower than the top two libraries, they still exhibit significant usage, contributing to a well-distributed library traffic pattern.

· The distribution of loans across different libraries provides insights into the varying levels of popularity and demand for library resources. Understanding these patterns allows for strategic resource allocation and improved library services.

· Libraries with higher circulation may consider expanding their collections, optimizing services, or implementing targeted promotions to meet the demand effectively.

· Libraries with lower circulation may benefit from promotional efforts, community engagement, or specific initiatives to attract more users and enhance their visibility.

· In summary, recognizing the libraries with the highest circulation enables informed decision-making for resource allocation, collection development, and service improvements to enhance user satisfaction and engagement across different libraries.

References:

https://app.diagrams.net/#G1UrxH-H_rD4iWfxeHE6ZHlGWjwb3x1gId

https://faker.readthedocs.io/en/master/

https://www.kaggle.com/datasets/ishikajohari/best-books-10k-multi-genre-data

https://github.com/laela27/e-library

Follow

Written by Laela Hajat Maqbullah

1 follower · 2 following

No responses yet

Write a response

What are your thoughts?

More from Laela Hajat Maqbullah

```
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   loan_id                   4269 non-null   int64
 1    no_of_dependents         4269 non-null   int64
 2    education                4269 non-null   object
 3    self_employed            4269 non-null   object
 4    income_annum             4269 non-null   int64
 5    loan_amount              4269 non-null   int64
 6    loan_term                4269 non-null   int64
 7    cibil_score              4269 non-null   int64
 8    residential_assets_value 4269 non-null   int64
 9    commercial_assets_value  4269 non-null   int64
 10   luxury_assets_value      4269 non-null   int64
```

Laela Hajat Maqbullah

Loan Approval Prediction: Leveraging Data Science for Automated Decision-Making

In this data science project, I aim to develop a loan approval prediction model to assist financial institutions in making faster and more...

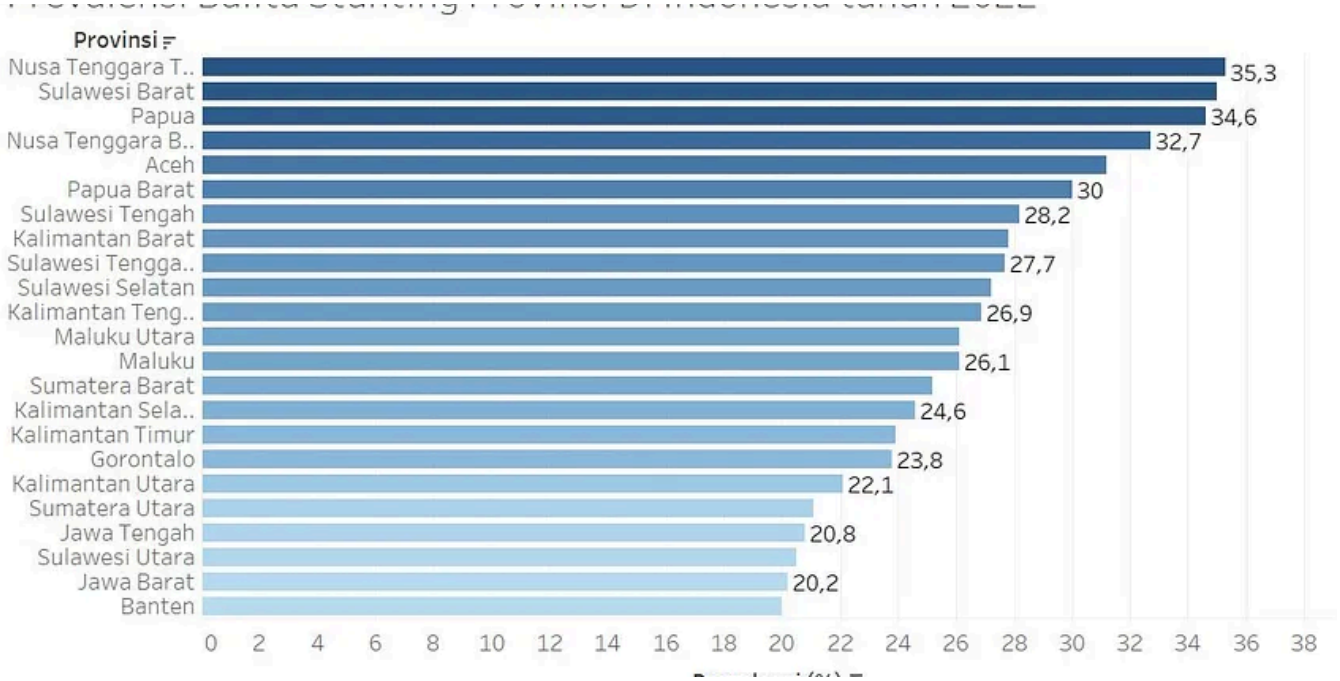
Oct 10, 2024

Laela Hajat Maqbullah

Introduction to Data Science

In today's world, where data holds a crucial role in various aspects of life and business, the term "data science" has become quite common...

May 10, 2024



Laela Hajat Maqbullah

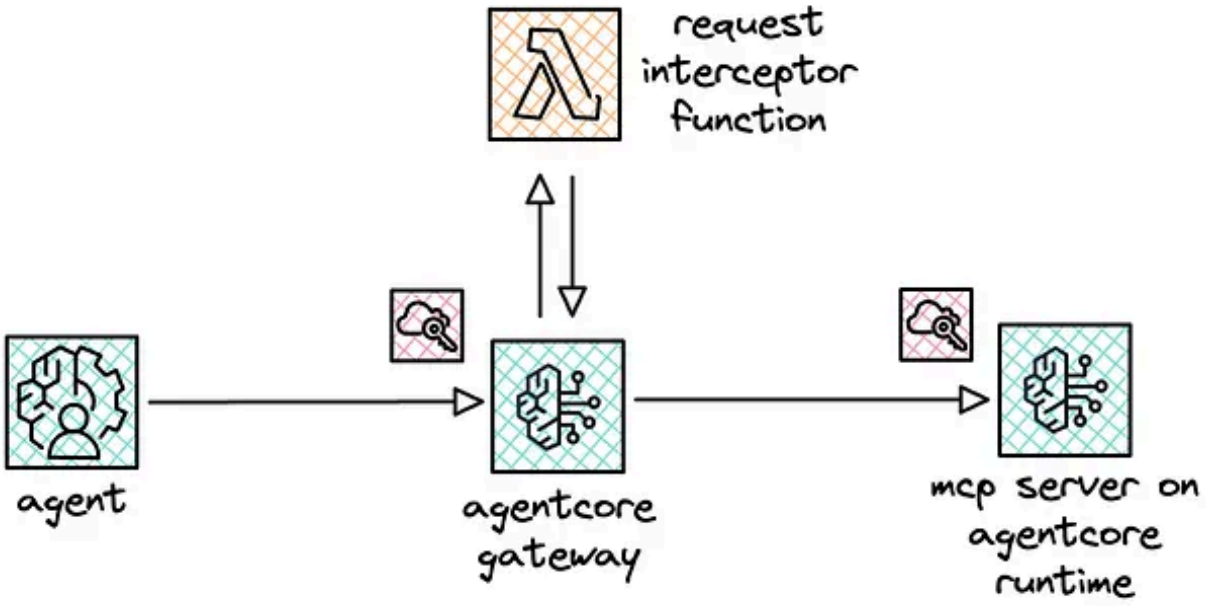
Analisis Prevalensi Stunting dan Kondisi Gizi Anak-anak di Jawa Barat — Sekolah Data Pacmann...

A. Background/Latar Belakang

Oct 1, 2023 1

See all from Laela Hajat Maqbullah

Recommended from Medium

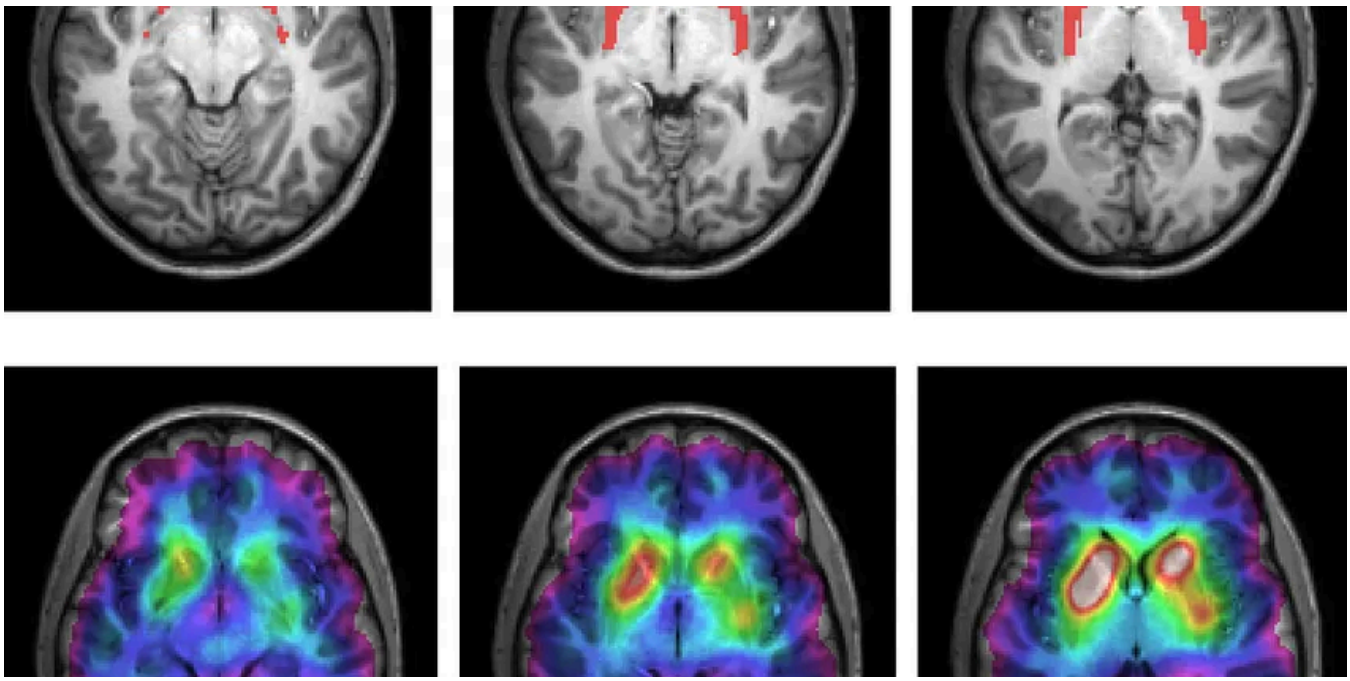


Heeki Park

Using spec-driven development with Claude Code

I no longer write code by hand. I wouldn't call myself a proper software development engineer by trade, nor have I deployed large-scale...

Feb 28 488 13

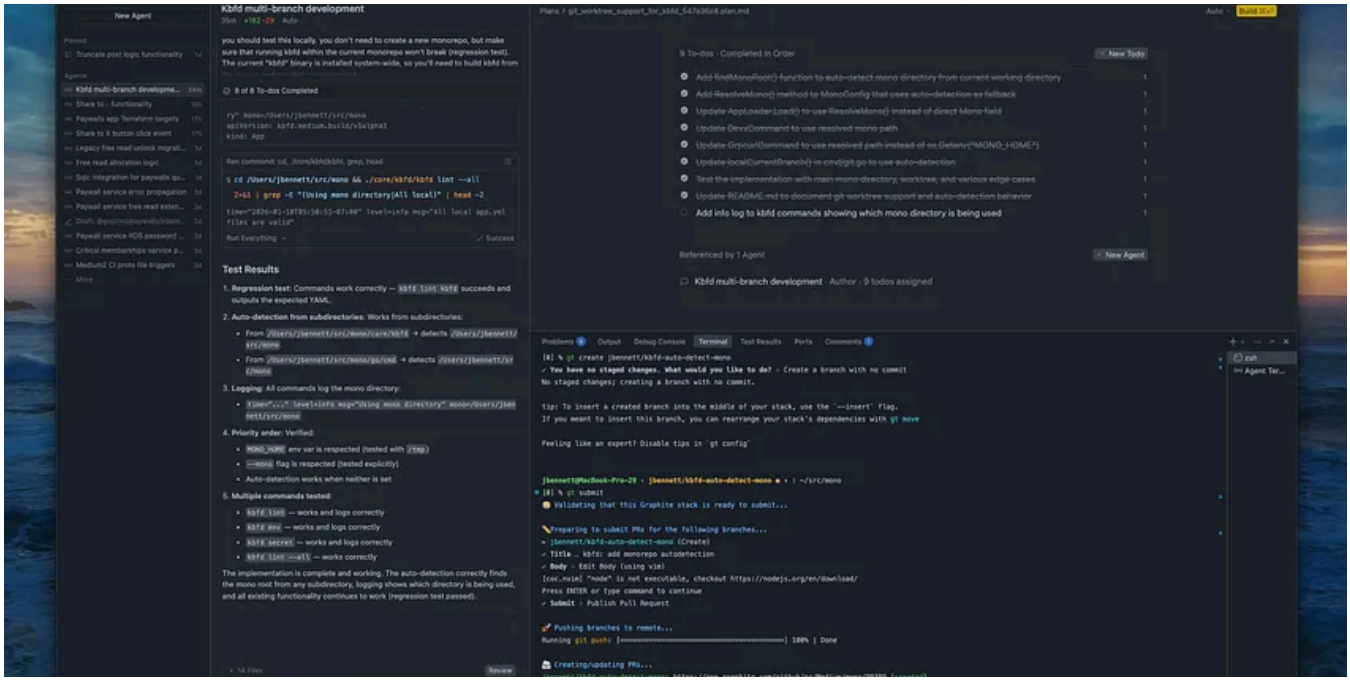


In Write A Catalyst by Dr. Patricia Schmidt

As a Neuroscientist, I Quit These 5 Morning Habits That Destroy Your Brain

Most people do #1 within 10 minutes of waking (and it sabotages your entire day)

Jan 14 38K 693



Jacob Bennett

The 5 paid subscriptions I actually use in 2026 as a Staff Software Engineer

Tools I use that are (usually) cheaper than Netflix

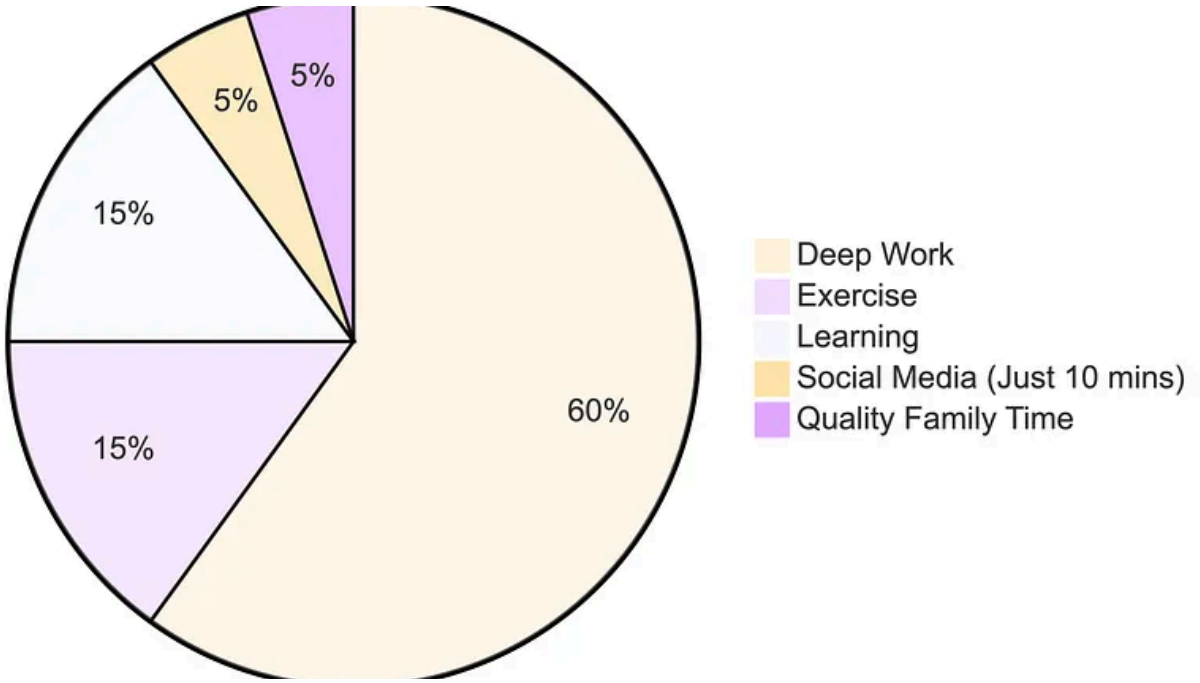
Jan 18 4K 92

Prashant Priyadarshi

Microsoft Most Frequent Low Level Design Questions from Recent Interviews

I am listing the top low level design questions that you will come across during Microsoft interviews. I have built this list from recent...

Sep 15, 2025 52

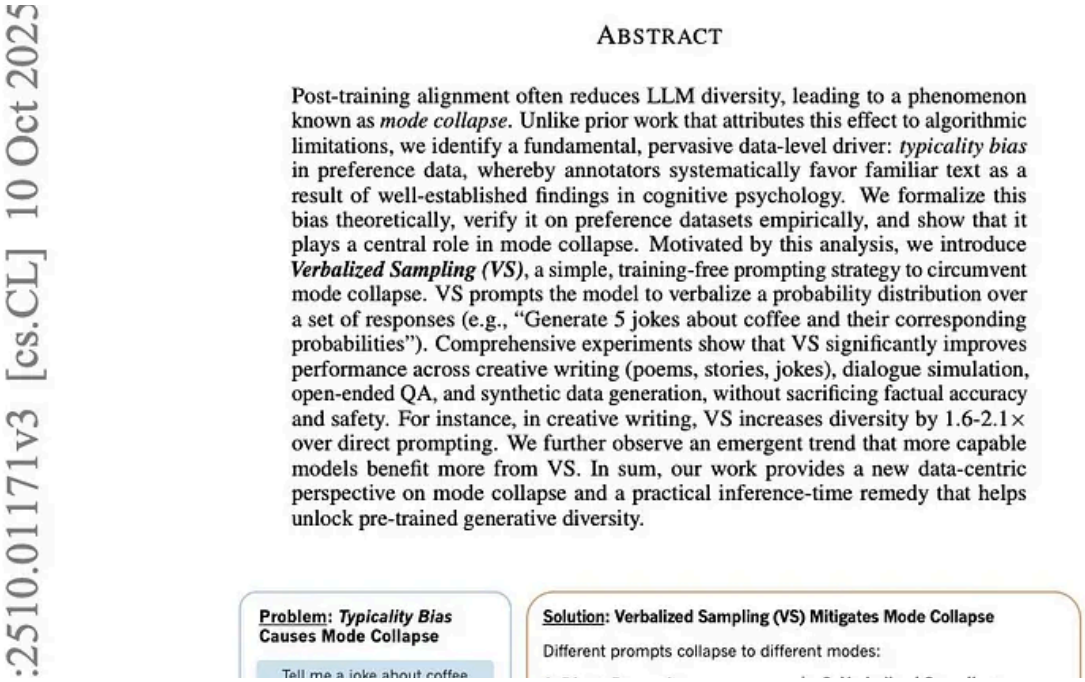


In Level Up Coding by Teja Kusireddy

I Stopped Using ChatGPT for 30 Days. What Happened to My Brain Was Terrifying.

91% of you will abandon 2026 resolutions by January 10th. Here's how to be in the 9% who actually win.

Dec 28, 2025 10.8K 384



:2510.01171v3 [cs.CL] 10 Oct 2025

ABSTRACT

Post-training alignment often reduces LLM diversity, leading to a phenomenon known as *mode collapse*. Unlike prior work that attributes this effect to algorithmic limitations, we identify a fundamental, pervasive data-level driver: *typicality bias* in preference data, whereby annotators systematically favor familiar text as a result of well-established findings in cognitive psychology. We formalize this bias theoretically, verify it on preference datasets empirically, and show that it plays a central role in mode collapse. Motivated by this analysis, we introduce ***Verbalized Sampling (VS)***, a simple, training-free prompting strategy to circumvent mode collapse. VS prompts the model to verbalize a probability distribution over a set of responses (e.g., "Generate 5 jokes about coffee and their corresponding probabilities"). Comprehensive experiments show that VS significantly improves performance across creative writing (poems, stories, jokes), dialogue simulation, open-ended QA, and synthetic data generation, without sacrificing factual accuracy and safety. For instance, in creative writing, VS increases diversity by 1.6-2.1× over direct prompting. We further observe an emergent trend that more capable models benefit more from VS. In sum, our work provides a new data-centric perspective on mode collapse and a practical inference-time remedy that helps unlock pre-trained generative diversity.

Problem: *Typicality Bias* Causes Mode Collapse

Tell me a joke about coffee

Solution: Verbalized Sampling (VS) Mitigates Mode Collapse

Different prompts collapse to different modes:

In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I Can't Believe It Worked)

ChatGPT keeps giving you the same boring response? This new technique unlocks 2× more creativity from ANY AI model—no training required...

Oct 19, 2025 25K 662

See more recommendations